

Optical Flow Estimation via Horn-Schunck and ε -Regularized PDEs

Victor Sorgi Pedroso

Contents

1	Problem Description	2
2	Mathematical Formulation	2
3	PDE Derivation	3
4	Discretization and Implementation	4
4.1	Spatial Derivatives	4
4.2	Temporal Derivative	4
4.3	Boundary Conditions	5
4.4	Time Step and Stability	5
5	Experimental Results	6
5.1	Simulated Data Error Calculation	6
5.2	Standard Horn-Schunck	6
5.3	ε -Augmented Horn-Schunck	7
5.4	Comparison	10
6	Discussion	10
7	Thank you!	11

1 Problem Description

Optical flow is a method for estimating how objects/pixels are moving between consecutive frames of a video. Given two frames that are taken at nearby points in time, optical flow tries to assign a motion vector to every pixel in the image, indicating both the direction and magnitude of its movement. The result of optical flow is a vector field which gets overlaid on the original frame, where each arrow points in the direction the pixel traveled, and the overall direction pointed to by the arrows shows the overall motion of the object.

The main challenge of optical flow is that motion cannot be directly observed from the data that comes with an image. Instead, all that we have is access to is the brightness at each pixel of the image in each of the analyzed frames. Optical flow therefore analyzes how these brightnesses change between frame and uses that change to infer motion. The problem with relying on brightness is that we need to assume that the overall brightness of different objects remains constant between frames, as if an object changes in brightness optical flow might interpret that as movement, when in reality all that changed was the lighting conditions. In practice this assumption does not fully hold up. A variety of different factors, such as shadows, lighting, and even changing camera settings can cause changes in brightness independent of motion.

Another difficulty with performing optical flow is when large regions are a consistent color/texture. If a shape has the same brightness value everywhere, the only region where motion can be estimated is the border of the shape as everything on the inside will appear as though it has not moved since each pixel might have shifted over, but it was replaced by one of the same brightness, therefore appearing to optical flow as though nothing is changing. To obtain a single unique solution different methods, such as the Horn-Schunck method explored in this paper, introduce a smoothness assumption which assumes that neighboring pixels tend to move similar to one another. This assumption allows us to find a solution that's computed across the entire image, even when the shape of interest is completely uniform.

2 Mathematical Formulation

Now that we have explained the optical flow problem, we need to find a way to mathematically describe what optical flow does. The first part to tackle is the brightness constancy assumption that actually lets us use optical flow for estimating motion. Mathematically, if a pixel at position (x, y) moves with velocity (u, v) , where our velocities are in pixels/frame, then $I(x + u, y + v, t + 1) = I(x, y, t)$, allows us to track a pixel between consecutive frames. By linearizing the resulting equation with a Taylor expansion we get the main optical flow constraint equation:

$$I_x u + I_y v + I_t = 0 \quad (1)$$

where I_x, I_y, I_t are the derivatives of our image (I) with respect to x, y, t respectively. In this equation $I_x u + I_y v$ is the mathematical way of representing the expected brightness change in the x and y directions, and I_t is the actual brightness change between two frames, so by summing those and equating them to zero we are saying that if we move in the right direction by the right amount, the brightness should appear to remain constant. The problem with this equation is that we only have one equation but two unknowns, u and v , so we have infinitely many solutions. So only using this constraint equation is not enough for optical flow and we need a secondary equation, so that we can actually get a single answer. This is where the energy function comes in.

As we previously described if we just assumed that the motion of every pixel is completely independent, we end up with an under determined equation, that does not have a single answer. This is where the smoothness term described in Section 1 comes in. One way to enforce smoothness is to penalize neighboring pixels with very different flow vectors. By keeping the gradients of u and v , $|\nabla u|^2$ and $|\nabla v|^2$, small we accomplish exactly that.

By combining our first constraint with the smoothness term we get the following energy function:

$$E(u, v) = \int_{\Omega} \lambda/2 (I_x u + I_y v + I_t)^2 + (1 - \lambda) (|\nabla u|^2 + |\nabla v|^2) d\Omega \quad (2)$$

Note the $1/2$ multiplying the lambda of the data term is there to cancel out the factor of 2 we get in the derivatives taken later in this paper. In this equation λ is a chosen parameter between 0 and 1 that helps

balance the two terms in the energy function. As λ gets close to one we trust what we are actually seeing happen more, and as it approaches 0 we more strongly enforce the smoothness term. The objective of optical flow will be to minimize this energy function, meaning we get a solution that simultaneously satisfies both terms as well as possible, meaning we get a solution that respects the observed brightness changes, as well as varies smoothly across the image.

One modification that we will be exploring in this project is the addition of an ε penalty parameter. The standard smoothness term, $|\nabla u|^2 = u_x^2 + u_y^2$, quadratically penalizes the gradients in the flow field, so large gradients are penalized much more heavily than small ones, leading to a very smooth flow field, including inside objects where flow is not actually happening. With the addition of ε our energy equation becomes

$$E(u, v) = \int_{\Omega} \lambda/2(I_x u + I_y v + I_t)^2 + (1 - \lambda) \left(\sqrt{u_x^2 + u_y^2 + \varepsilon^2} + \sqrt{v_x^2 + v_y^2 + \varepsilon^2} \right) d\Omega \quad (3)$$

where the smoothness less aggressively penalizes large gradients, so our flow field is allowed to have sharp transitions at the boundaries of motions, which is a lot more realistic.

3 PDE Derivation

One way of ensuring we are minimizing the energy function previously described is by applying the Euler-Lagrange equation:

$$L_u - \frac{\partial}{\partial x} L_{u_x} - \frac{\partial}{\partial y} L_{u_y} = 0 \quad (4)$$

From the energy function our L is everything inside the integral, so in our case:

$$L = \lambda/2(I_x u + I_y v + I_t)^2 + (1 - \lambda)(|\nabla u|^2 + |\nabla v|^2) \quad (5)$$

We start off by applying the Euler-Lagrange to the standard Horn-Schunck equation, first by calculating L_u , L_{u_x} , and L_{u_y} :

$$L_u = \lambda(I_x u + I_y v + I_t)I_x \quad (6)$$

$$L_{u_x} = (1 - \lambda)u_x \quad (7)$$

$$L_{u_y} = (1 - \lambda)u_y \quad (8)$$

Which after plugging into the Euler-Lagrange formula gives us the following equation:

$$\lambda(I_x u + I_y v + I_t)I_x - (1 - \lambda)(u_{xx} + u_{yy}) = 0 \quad (9)$$

To convert this into a gradient descent PDE, we must introduce τ , an artificial time parameter, and then our gradient descent PDE $u_{\tau} = -\nabla_u E$:

$$u_{\tau} = -\lambda(I_x u + I_y v + I_t)I_x + (1 - \lambda)(u_{xx} + u_{yy}) \quad (10)$$

Following the same process but taking derivatives with respect to v instead of u , by symmetry we obtain the second gradient descent PDE:

$$v_{\tau} = -\lambda(I_x u + I_y v + I_t)I_y + (1 - \lambda)(v_{xx} + v_{yy}) \quad (11)$$

Together, these two PDEs form the standard Horn-Schunck optical flow system, which we evolve iteratively until convergence to obtain our flow field (u, v) .

For the ε -augmented version, the data term is identical so there are no changes to L_u . However, the smoothness term L_{u_x} now becomes:

$$L_{u_x} = (1 - \lambda) \frac{u_x}{\sqrt{u_x^2 + u_y^2 + \varepsilon^2}} \quad (12)$$

To obtain the full Euler-Lagrange equation we must compute $\frac{d}{dx}L_{u_x}$ and $\frac{d}{dy}L_{u_y}$ using the product and chain rules, then add them together. This gives the following divergence term:

$$\nabla \cdot \left(\frac{\nabla u}{\sqrt{u_x^2 + u_y^2 + \varepsilon^2}} \right) = \frac{u_{xx} + u_{yy}}{\sqrt{u_x^2 + u_y^2 + \varepsilon^2}} - \frac{u_x^2 u_{xx} + 2u_x u_y u_{xy} + u_y^2 u_{yy}}{(u_x^2 + u_y^2 + \varepsilon^2)^{3/2}} \quad (13)$$

Applying gradient descent as before gives the final ε -augmented PDE:

$$u_\tau = -\lambda(I_x u + I_y v + I_t)I_x + (1 - \lambda) \left(\frac{u_{xx} + u_{yy}}{\sqrt{u_x^2 + u_y^2 + \varepsilon^2}} - \frac{u_x^2 u_{xx} + 2u_x u_y u_{xy} + u_y^2 u_{yy}}{(u_x^2 + u_y^2 + \varepsilon^2)^{3/2}} \right) \quad (14)$$

And by symmetry the corresponding PDE for v is:

$$v_\tau = -\lambda(I_x u + I_y v + I_t)I_y + (1 - \lambda) \left(\frac{v_{xx} + v_{yy}}{\sqrt{v_x^2 + v_y^2 + \varepsilon^2}} - \frac{v_x^2 v_{xx} + 2v_x v_y v_{xy} + v_y^2 v_{yy}}{(v_x^2 + v_y^2 + \varepsilon^2)^{3/2}} \right) \quad (15)$$

4 Discretization and Implementation

To numerically solve these PDEs we need to discretize all the spatial and temporal derivatives using finite differences. As we are looking at images we work on a grid of pixels where each pixel is separated by a distance of $\Delta x = \Delta y = 1$. For the description of the finite differences we will be using generic x , y , and t for the spatial and temporal coordinates.

4.1 Spatial Derivatives

For our first order spatial derivatives I_x and I_y we use central differences:

$$I_x(i, j) = \frac{I(i, j+1) - I(i, j-1)}{2} \quad (16)$$

$$I_y(i, j) = \frac{I(i+1, j) - I(i-1, j)}{2} \quad (17)$$

We chose to use central differences over forward or backward differences because they are accurate to the second order, meaning that our error scales with Δx^2 rather than Δx . Being a central difference it also doesn't bias the estimate towards one side of the pixel, so the estimate is actually centered on the pixel of interest, not some other pixel.

For our second order derivatives, I_{xx} and I_{yy} , we once again use a central difference, but this time it is a second order central difference:

$$I_{xx}(i, j) = I(i, j+1) - 2I(i, j) + I(i, j-1) \quad (18)$$

$$I_{yy}(i, j) = I(i+1, j) - 2I(i, j) + I(i-1, j) \quad (19)$$

Finally for the mixed derivative I_{xy} needed in the ε -augmented version we use:

$$I_{xy}(i, j) = \frac{I(i+1, j+1) - I(i+1, j-1) - I(i-1, j+1) + I(i-1, j-1)}{4} \quad (20)$$

4.2 Temporal Derivative

The temporal derivative I_t is a lot simpler as it is simply the difference between the two frames, so there is no need to use any approximation scheme.

$$I_t(i, j) = I_2(i, j) - I_1(i, j) \quad (21)$$

4.3 Boundary Conditions

At the borders of the image, the central difference needs to look at pixels that are outside of the image. There are many ways to handle this, but for this project we chose to zero pad the images. Zero-padding just means we add a layer of zeros around the entire image, so when we perform the central difference at the original border of the image, it now has a value to use, even though originally there wasn't anything there. This is generally a safe way of doing it as motion near the edge of the image is rarely the motion that is actually being focused on.

4.4 Time Step and Stability

Both PDEs are evolved using an explicit forward Euler scheme, where our previous value, u^n is added to our gradient descent PDE multiplied by our time step giving us the new value, u^{n+1} . This is given in the following equation:

$$u^{n+1} = u^n + \Delta\tau \cdot (-\lambda(I_x u + I_y v + I_t)I_x + (1 - \lambda)(u_{xx} + u_{yy})) \quad (22)$$

For the standard Horn-Schunck scheme the stability condition requires:

$$\Delta\tau \leq \frac{\Delta x^2}{4 * (1 - \lambda)} \quad (23)$$

As $(1 - \lambda)$ is always less than 1, using any time step less than 0.25 is a safe bet, and for this paper we use $\Delta\tau = 0.001$ which guarantees we fall well within this limit. For the ε -augmented version the effective diffusion coefficient is $\frac{1-\lambda}{\sqrt{u_x^2 + u_y^2 + \varepsilon^2}}$, giving a stability condition of approximately:

$$\Delta\tau \leq \frac{\varepsilon^2}{4} \quad (24)$$

This is a lot stricter than the standard Horn-Schunck condition as small ε values quickly lead to us having to use very small steps, due to the square dependence on it. We decided to keep the same time step of $\Delta\tau = 0.001$, so we chose $\varepsilon \geq 0.1$ to ensure stability with the chosen time step. Violating these stability conditions causes the solution to blow up, so instead of converging, at each iteration pixel values amplify each other. As an example of this, Figure 1 shows the result of applying the heat equation to diffuse a square, with a stable and unstable time step.

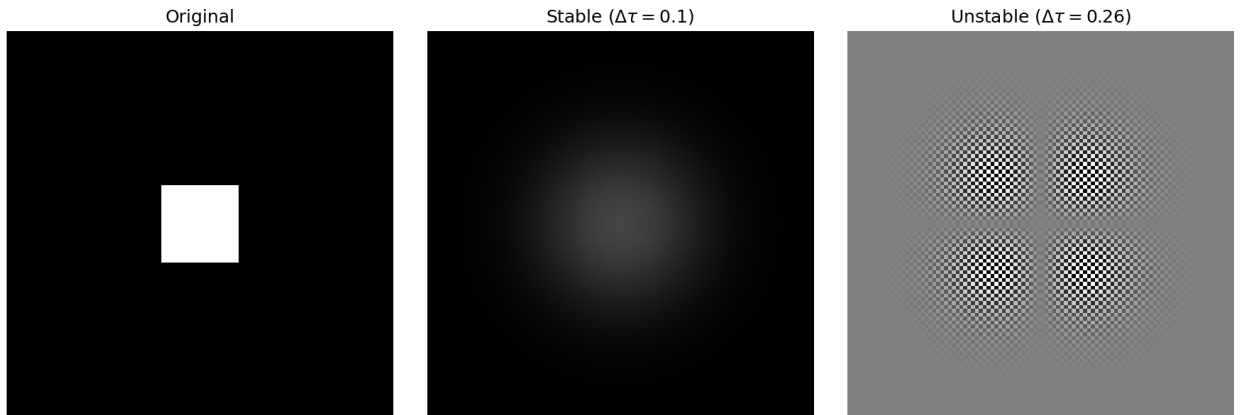


Figure 1: Left: original image. Middle: stable diffusion ($\Delta\tau = 0.1$). Right: unstable diffusion ($\Delta\tau = 0.26$), showing characteristic checkerboard instability.

5 Experimental Results

5.1 Simulated Data Error Calculation

To test our implementation we use a generated 100×100 image containing a white square on a black background. To get the white square we set every pixel in the range $[40 : 60, 40 : 60]$ to white in the first frame, and for our second frame we shift it by one pixel in both the x and y directions, so our white square is at $[41 : 61, 41 : 61]$. This gives us a known ground truth where the flow is $(u, v) = (1, 1)$ along the border of the square and zero everywhere else.

To quantitatively evaluate our results we use the mean error, defined as:

$$\text{Error} = \frac{1}{N} \sum_{i,j} \sqrt{(u(i,j) - u_{\text{true}}(i,j))^2 + (v(i,j) - v_{\text{true}}(i,j))^2} \quad (25)$$

where the sum is taken over all pixels. We set u_{true} and v_{true} to be 1 at the pixels at the border of the square and to be zero everywhere else. Because of this, our error measures both how well the method estimates flow at the border and how much it spreads flow into regions where there should be none.

5.2 Standard Horn-Schunck

We begin by applying the standard Horn-Schunck method to our test images for 5 different values of λ to understand how the balance between the data and the smoothness terms affects our results.

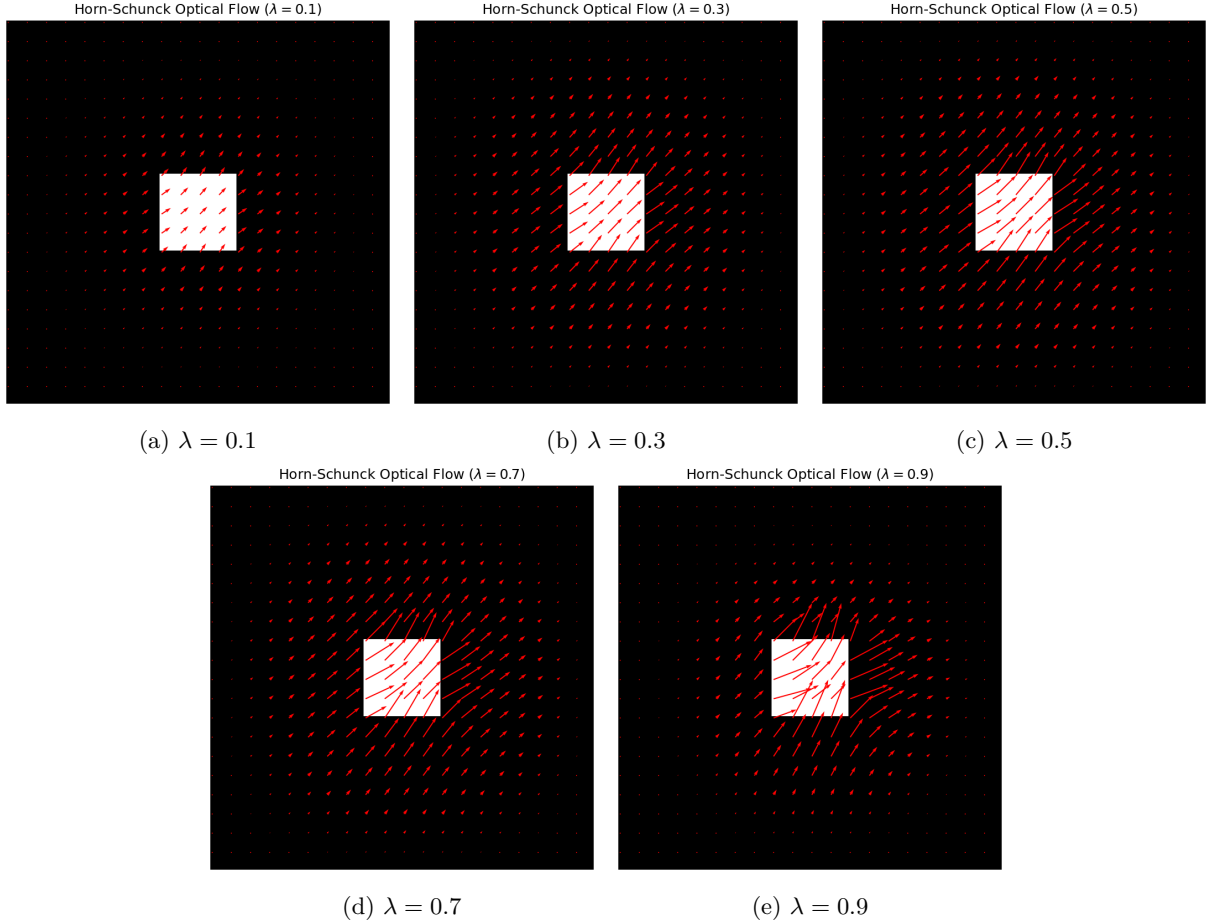


Figure 2: Standard Horn-Schunck optical flow fields for different λ values

λ	Iterations	Error
0.1	69,493	0.1093
0.3	88,804	0.2171
0.5	104,241	0.2591
0.7	131,507	0.2678
0.9	188,405	0.2152

Table 1: Mean error and iterations to convergence for standard Horn-Schunck across different λ values. Convergence is defined as $\max_{i,j} |u^{n+1}(i,j) - u^n(i,j)| < 10^{-6}$.

From Table 1, we can see that error is lowest at $\lambda = 0.1$ and generally increases as λ increases, with a slight drop at $\lambda = 0.9$. At $\lambda = 0.1$ the smoothness term completely dominates, which forces the flow field to be almost zero across the entire image. This is exactly why the error is so low, since our ground truth is zero almost everywhere, such a low λ is getting the correct answer in terms of error, but when looking at the actual flow field we see that it is completely missing the actual motion at the border. At $\lambda = 0.9$ our flow is almost fully data driven, meaning the method is strongly responding where brightness changes are actually occurring, which is exactly at the border of the square. Unlike $\lambda = 0.1$, this relatively low error is achieved by correctly estimating the motion rather than simply producing a near-zero flow field everywhere. Looking at the quiver plot, the arrows are more concentrated at the border and generally pointing in the correct diagonal direction, making this the most physically accurate result despite not having the lowest error.

In the middle, $\lambda = 0.5$ produces the highest error overall. Even though we are partially data driven which would correctly generate flow at the border, the smoothness term is also strong enough to spread it out across the background. This combination gives us the worst of both worlds, we get all the flow extending into the background from the smoothness term, while the data term is not strong enough to fully concentrate it at the border. So even though this plot might look the most visually balanced, an equal combination of data driven and smoothness actually produces the least accurate result for this test case.

5.3 ε -Augmented Horn-Schunck

Next we applied the augmented Horn-Schunck to the same images, but this time we keep $\lambda = 0.5$, and test it for 8 different values of ε . The reason we keep λ constant is that we want to isolate the effect of ε on our results, and $\lambda = 0.5$ is the natural choice as it gives equal weights to the data and smoothness terms, so neither dominates. This makes it a fair starting point for comparing the different ε values.

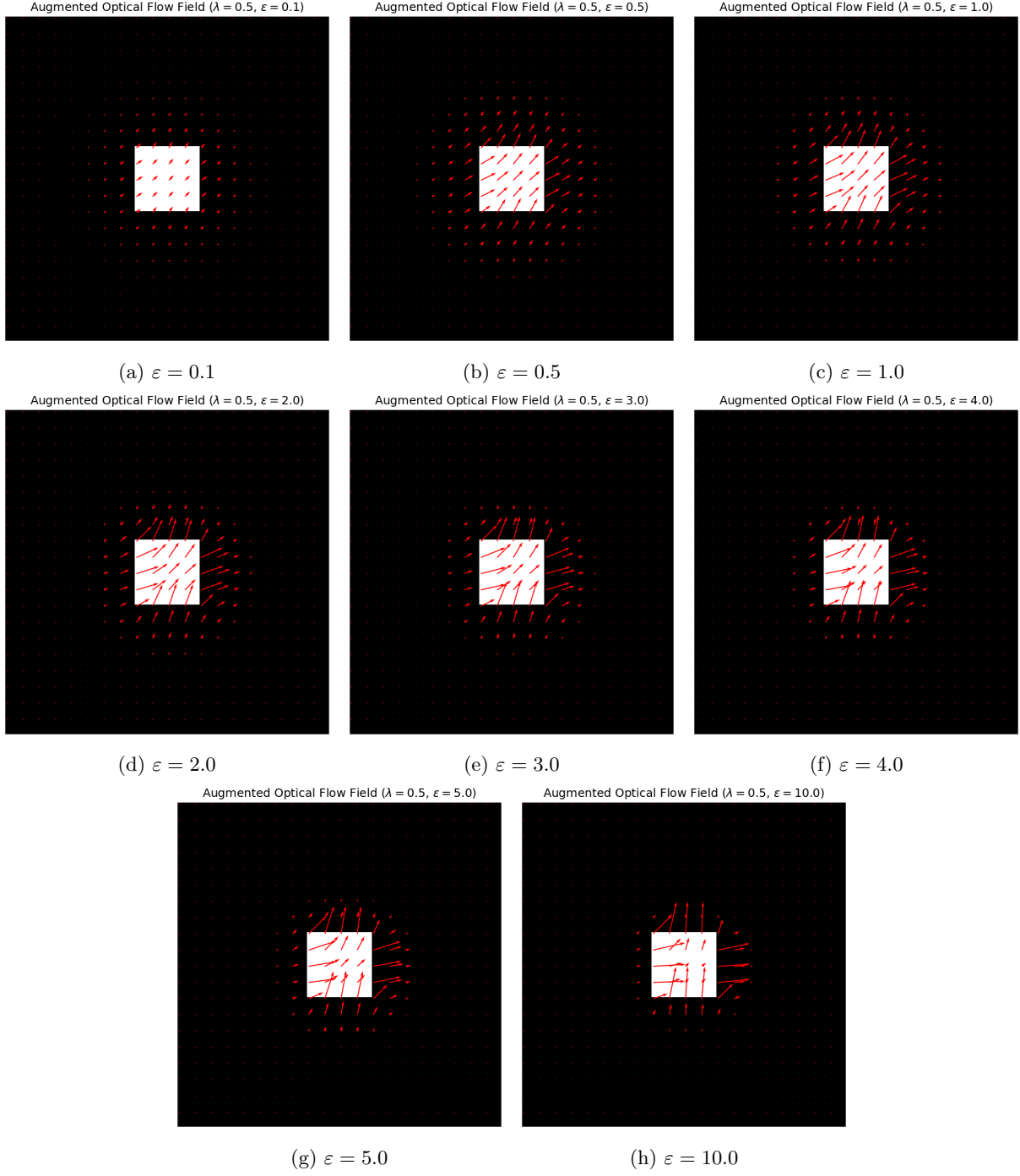


Figure 3: Optical flow fields for different ϵ values with $\lambda = 0.5$

ε	Iterations	Error
0.1	79,387	0.0797
0.5	172,340	0.1167
1.0	222,043	0.1144
2.0	285,280	0.1059
3.0	324,495	0.0985
4.0	348,008	0.0922
5.0	359,945	0.0865
10.0	368,244	0.0686

Table 2: Mean error and iterations to convergence for the ε -augmented Horn-Schunck method across different ε values with $\lambda = 0.5$. Convergence is defined as the maximum change in u between consecutive iterations falling below 10^{-6} , i.e. $\max_{i,j} |u^{n+1}(i,j) - u^n(i,j)| < 10^{-6}$.

For the ε -augmented method we see three distinct behaviors begin to emerge, which helps us better understand what this parameter is actually doing.

For the smallest value we tested, $\varepsilon = 0.1$, the flow field is extremely even and small, behaving similar to the previous case with $\lambda = 0.1$. Despite having the lowest error, this result is the least physically meaningful, and has a small error for pretty much the same reason as the small error in $\lambda = 0.1$, where the flow field is almost zero across the entire image, which matches our ground truth very closely, once again getting the “right” answer for the wrong reasons.

For the middle range of values, those being $\varepsilon = 1.0$ to $\varepsilon = 5.0$, the results are visually the most accurate. The arrows are mostly concentrated around the borders of the white square and mostly point in the correct diagonal direction. We do see some arrows inside the square, but the flow is minimally spread outside the square, being significantly less spread out than the standard Horn-Schunck results. The smoothness term here is strong enough to propagate directional information around the corners of the square, making it so the arrows actually point diagonally rather than simply up or right depending on which face of the square they are on. These results represent the best balance between the data and smoothness terms.

For the largest value tested, $\varepsilon = 10.0$, the flow becomes quite noisy and inconsistent. We once again get a relatively low error since the smoothness term is almost completely turned off, concentrating the arrows on the border of the square. However the smoothness is so weak that each side of the square is essentially being treated as though it is moving independently. This can be seen in the plot where the arrows are almost entirely split between being completely vertical or completely horizontal, with very few actually showing the true diagonal motion of the square.

The iteration count in Table 2 also reveals an interesting trend that was opposite to what was originally expected. We initially assumed that lower ε values would require the most iterations to converge, however the large ε values actually required significantly more. $\varepsilon = 0.1$ converges in roughly 79,000 iterations whereas $\varepsilon = 10.0$ takes over 350,000 iterations to converge. After thinking about this more it makes sense, the larger ε weakens the effective diffusion of the method, slowing the spread of information across the image and requiring significantly more iterations to reach steady state. So there is a clear tradeoff when choosing ε , where the values that produce the best visual results also require a lot more computation than smaller ε values.

5.4 Comparison

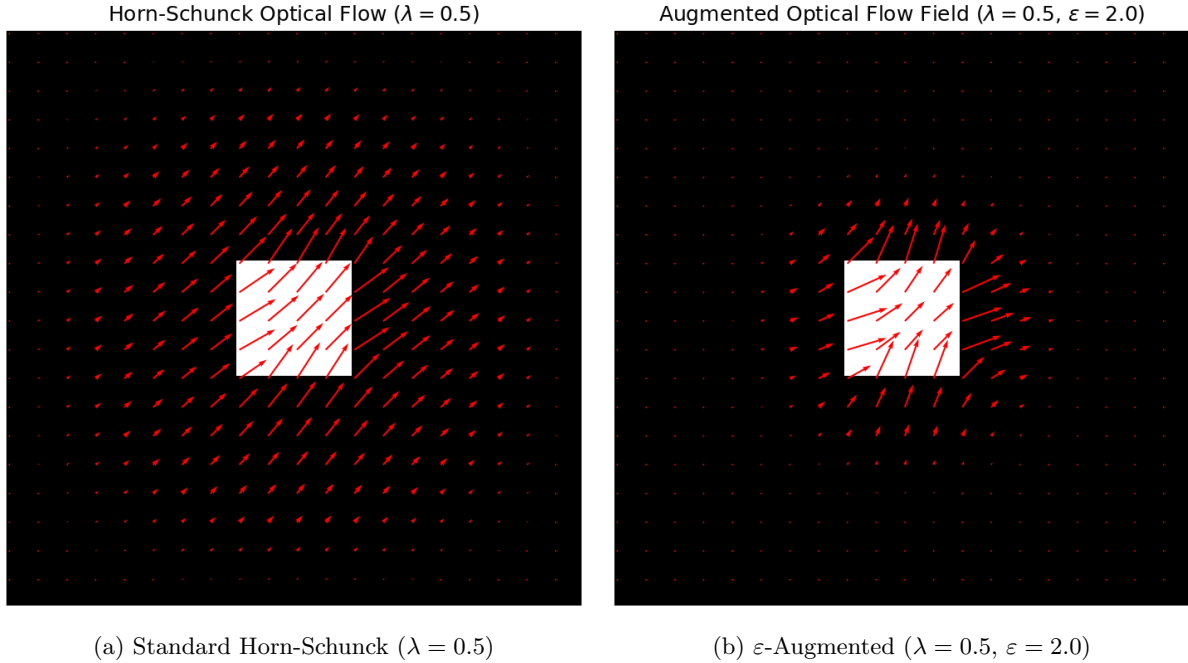


Figure 4: Comparison between standard Horn-Schunck and ε -augmented Horn-Schunck at $\lambda = 0.5$

Figure 4 shows a comparison between the standard Horn-Schunck and the ε augmented method, both at $\lambda = 0.5$. The clearest difference between the two images is how far outside the square our flow spreads. The standard approach has flow that reaches almost twice as far as the augmented method. This shows how much better the augmented method is at limiting how far smoothness spreads the flow, keeping it closer to where it is actually happening. For the direction of flow both methods successfully find the diagonal motion of the square, and don't really show any significant differences between the standard and augmented method.

In terms of error, the best standard Horn-Schunck result achieves an error of 0.1093 at $\lambda = 0.1$, while the best ε -augmented result achieves 0.0797 at $\varepsilon = 0.1$. However, as was discussed in a previous section, these low errors are achieved for the wrong reasons, both of the extremely low errors were achieved by making the flow field close to zero across the entire image, instead of actually properly getting the flow from the image. Apart from these two extreme cases, the errors generated by the augmented method were about half that of the standard, most likely due to the lessened spread that we get in the augmented case. As the entire background should have a flow of zero, having any sort of flow extend into the background greatly increases the error, which is exactly what we see happen for the standard Horn-Schunck.

The key tradeoff between the two models is the cost for running them. The standard method converges in roughly 70,000 to 190,000 whereas the ε augmented method takes up to 370,000 iterations to converge. So if accuracy is critical then the additional computational cost may be justified, but if speed matters more, and accuracy is not as important then the speed of the standard method might be more beneficial.

6 Discussion

Throughout doing this project I was able to learn a lot more about optical flow, beyond the foundations that we learned about in class, implementing this gave me having a much deeper understanding of how everything works.

First it was interesting to actually see the effect that ε augmentation has on optical flow. The biggest thing

I saw is that replacing the quadratic smoothness keeps the flow a lot more contained to where we actually have the motion happening, but at the same time it does not have a large effect on the flow on the inside of the square. I understand that theoretically the flow there should be 0, but even the largest ε values we tested still show significant flow on the inside of the square.

Next it was interesting to learn more about the actual effects that λ and ε have on the flow fields. They both have a sweet spot and initially I assumed that it would be easy to find what works best by simply looking at the error. However, the results of this experiment showed that the error can be completely misleading, since it can basically fall into a false minimum where the error appears small, but not because it is properly measuring the flow, but just because the flow field is becoming extremely small everywhere.

I also learned that the ε comes with significant increases in computational requirements. I initially thought that there would be no reason to use the standard version, because from what we learned in class it appeared as though the augmented version would lead to significant improvements to the flow fields, but from this experiment I now understand that the benefits of each are a lot more dependent on use case, instead of being a clear augmented is better and standard is worse.

In the future it would be interesting to look at some other ways of calculating error so that it is less affected by the extreme cases that we saw in this paper, and is a more useful metric of how well our optical flow is actually performing. It would also be interesting to further investigate what different values of ε are actually good at to better understand when it is worth dealing with the higher iteration count and when it isn't. Finally this method works well for single pixel displacements as our central difference looks one pixel in each direction, but it would be interesting to explore ways of expanding this work to function well for larger gaps, whether that means using more points in the central difference or some other approach, it would provide valuable insight into how this can become more useful for real world applications.

7 Thank you!

Hi Dr. Yezzi, I wanted to add something to the end of this project just to thank you for this class. I don't remember the last time where I came out of a class feeling like I actually learned a lot. The class was organized perfectly and your teaching combined with the assignments made me actually want to learn, and also did an amazing job of helping us learn without just giving everything away. So genuinely thank you for the semester, and I appreciate all the time and effort you put into making sure this class isn't just about memorizing a bunch of random equations, but actually ensuring that we understand what we are learning.